Benchmarking DAv3c on the Max-Cut Problem:

Separating Hardware Performance from Algorithmic Efficiency

Sargunpreet Kaur
(639134)

Master's Research Project

Supervised by:

Prof. Dr. Martin Kliesch
Salwa Shaglel

TUHH

Institute for Quantum Inspired and Quantum Optimization
Hamburg University of Technology, Germany

July 2, 2026

1 Introduction

Combinatorial optimization (CO) problems arise in a wide range of real-world applications, including network design, scheduling, finance, and machine learning [1]. Many of these problems are NP-hard and are therefore unlikely to admit polynomial-time solutions, even with quantum computing approaches [2]. This inherent computational complexity motivates the development of heuristic solutions methods that trade optimality guarantees for scalability and efficiency.

A large class of CO problems, including Max-Cut, can be formulated as Quadratic Unconstrained Binary Optimization (QUBO) problems [3]. By expressing diverse optimization problems within a common mathematical framework, QUBO formulations enable the application of a broad range of solution approaches, including classical heuristics, metaheuristics, and hardware-based or quantum-inspired optimization methods [4]. These developments have significantly expanded the set of practical approaches for solving large and complex instances.

Among such methods, Fujitsu's Digital Annealer (DA) has demonstrated strong empirical performance on CO tasks. Recent benchmark studies, such as *A comprehensive benchmark of an Ising machine on the Max-Cut problem* [5], compare successive generations of the DA (DAv2 and DAv3) with state-of-the-art classical heuristics on large-scale Max-Cut instances from the MQLib repository [6]. These studies show that DA-based solvers achieve competitive, and often superior solution quality within comparable runtime limits, particularly when evaluated across graph size and density categories. However, the reported performance reflects not only the underlying optimization algorithm, but also the advantages provided by specialized hardware acceleration.

This observation raises two key questions. First, to what extent can the performance of DA-based approaches be attributed to the optimization algorithm itself, rather than to hardware-specific acceleration? Second, beyond graph size and density, how do additional instance characteristics—including structural graph properties and edge-weight structure—influence solver performance and solution quality across different optimization strategies?

To address the first question, we evaluate a CPU-based implementation of the DA algorithm, referred to as DAv3c. By removing hardware-specific acceleration, DAv3c enables a more direct assessment of the algorithmic contribution of the DA framework. Using the same benchmark dataset and evaluation methodology as in Shaglel et al. [5], we compare DAv3c against the best-performing classical heuristics to evaluate the overall competitiveness of the solver on large-scale Max-Cut instances, and further analyze its performance relative to DAv2 and DAv3 where applicable.

To address the second question, we additionally investigate how solver behavior changes under different graph and edge-weight characteristics. In particular, we analyze solver performance across multiple structural and edge-weight-related properties, including bipartite and odd-cycle structures, edge-weight representation (integer versus non-integer weights), dominant edge-weight concentration, and optimality behavior on specialized graph subsets. These analyses aim to identify whether specific optimization strategies exhibit systematic advantages under particular landscape characteristics induced by graph topology or edge-weight distributions.

2 Background and Methodology

2.1 Max-Cut and QUBO Formulation

The Max-Cut problem is a well-known combinatorial optimization problem—specifically, one of Karp's 21 NP-complete problems [7]—defined on an undirected graph $G = (V, E)$, where V is the set of vertices and E is the set of edges. Each edge $(i, j) \in E$ is associated with a weight w_{ij}. The objective is to partition the vertex set V into two disjoint subsets such that the total weight of the edges crossing the partition is maximized. While the decision version of Max-Cut is NP-complete, its optimization variant is NP-hard.

To formulate this problem, we associate a binary decision variable $x_i \in \{0, 1\}$ with each vertex $i \in V$. The value of x_i indicates the subset to which vertex i is assigned. Two vertices i and j are in different subsets if $x_i \neq x_j$, and in the same subset if $x_i = x_j$.

Using this representation, the Max-Cut objective can be written as:

$$f(x) = \sum_{(i,j)\in E} w_{ij} \left(x_i + x_j - 2x_i x_j\right), \tag{1}$$

where the term $(x_i + x_j - 2x_i x_j)$ evaluates to 1 when $x_i \neq x_j$ (i.e., the edge crosses the cut) and 0 otherwise.

QUBO Formulation

The QUBO problems are typically expressed in the form:

$$\min\ x^\top Q x, \quad \text{subject to } x \in \{0, 1\}^n, \tag{2}$$

where $Q \in \mathbb{R}^{n\times n}$ is a (typically upper triangular) matrix encoding linear and quadratic coefficients. In general, QUBO is NP-hard [8] and can represent a wide class of CO problems, including Max-Cut.

Since the Max-Cut problem is naturally formulated as a maximization problem, while QUBO is conventionally defined as a minimization problem, we convert it by negating the objective:

$$\min\ -f(x). \tag{3}$$

Expanding the Max-Cut objective,

$$f(x) = \sum_{(i,j)\in E} w_{ij} x_i + \sum_{(i,j)\in E} w_{ij} x_j - \sum_{(i,j)\in E} 2w_{ij} x_i x_j, \tag{4}$$

and regrouping terms, we obtain:

$$f(x) = \sum_{i\in V} \left(\sum_{j\in N(i)} w_{ij} \right) x_i \ - \sum_{(i,j)\in E} 2w_{ij} x_i x_j, \tag{5}$$

where $N(i)$ denotes the set of neighbors of vertex i.

From this, the equivalent QUBO formulation $\min x^\top Q x$ is obtained by setting the coefficients of Q as:

$$Q_{ii} = -\sum_{j \in N(i)} w_{ij}, \tag{6}$$

$$Q_{ij} = 2w_{ij}, \quad \text{for } i < j \text{ and } (i,j) \in E, \tag{7}$$

$$Q_{ij} = 0, \quad \text{otherwise.} \tag{8}$$

Here, the diagonal entries Q_{ii} correspond to linear terms, while the off-diagonal entries Q_{ij} encode pairwise interactions between variables. The matrix Q is typically stored in upper triangular form without loss of generality.

This transformation shows that the Max-Cut problem can be expressed as a QUBO problem with $|V|$ binary variables and no explicit constraints. In practice, a small amount of preprocessing is applied: one variable is fixed to break the inherent symmetry of the problem (since swapping the two partitions yields an equivalent solution), and the variable with the largest linear coefficient is typically chosen for this purpose. Additionally, disconnected variables are removed, as they do not contribute to the objective. As a result, the resulting QUBO formulation generally involves $|V| - 1$ variables, or slightly fewer in the presence of isolated nodes.

Importantly, this reduction incurs only minimal overhead, as the Max-Cut objective is already quadratic in binary variables and can be directly mapped to the QUBO form with only minor transformations. This makes Max-Cut a representative problem that captures the core computational challenges of QUBO, while enabling the application of a wide range of optimization techniques, including classical heuristics, annealing-based methods, and quantum-inspired algorithms within a unified framework.

2.2 DA and Heuristic Solvers

In this work, we evaluate the performance of a CPU-based implementation of Fujitsu's DA, referred to as DAv3c, and compare it against state-of-the-art classical heuristics. To contextualize these results, we also compare its performance against previously reported benchmarks by Shaglel et al. [5] for the hardware-based DA variants, DAv2 and DAv3.

2.2.1 DA Framework

The DA is a quantum-inspired optimization architecture designed to solve QUBO problems using classical CMOS hardware [9]. In particular, it leverages highly parallel stochastic updates to evaluate and explore multiple candidate configurations simultaneously, enabling more efficient traversal of the solution space and improved ability to escape local optima compared to conventional sequential heuristics. In this sense, the DA can be viewed as a specialized Ising machine tailored for large-scale CO.

At its core, the DA operates on QUBO formulations using an annealing-based metaheuristic grounded in Markov chain Monte Carlo (MCMC), specifically the

Metropolis–Hastings algorithm [10]. Starting from an initial configuration x, candidate solutions x' are generated via bit flips and accepted with probability

$$p = \min\left\{1, \exp\left(-\frac{\Delta E}{T}\right)\right\}, \tag{9}$$

where $\Delta E = E(x') - E(x)$ and T is the temperature. This probabilistic acceptance enables the algorithm to escape local minima, while a gradual decrease in temperature according to a suitable cooling schedule [11] shifts the search from exploration to exploitation.

A distinguishing feature of the DA is its massive parallelism. Unlike conventional simulated annealing, where candidate updates are typically evaluated sequentially, the DA evaluates energy differences for all variables in parallel and employs a selection mechanism to transition between states. This results in an efficient approximate rejection-free update scheme that significantly accelerates convergence [12, 13].

The core hardware component of the DA is the Digital Annealer Unit (DAU), an application-specific integrated circuit (ASIC) implemented using CMOS technology. The DAU natively supports fully connected QUBO problems with up to 8,192 binary variables and operates using integer arithmetic. In particular, it supports 64-bit signed integers for quadratic terms and up to 76-bit signed integers for linear terms. For larger instances exceeding 4,096 variables, DAv2 reduces precision to 16-bit integers. When QUBO instances contain floating-point coefficients, an automatic scaling and rounding procedure is applied to map them into the supported integer range. Unlike many quantum and quantum-inspired approaches that require embedding due to limited connectivity, the DA directly supports arbitrary dense graph structures.

Different generations of the DA incorporate increasing levels of algorithmic sophistication:

- **DAv2:** Implements a simulated annealing framework capable of executing up to 128 independent runs in parallel across 16 hardware threads within a single DAU, with the best solution across all runs reported as the final output. It also includes optional support for parallel tempering and incorporates dynamic energy offsetting, where an offset is applied to the energy difference to increase acceptance probabilities when the search stagnates, particularly at low temperatures. This mechanism helps the algorithm escape local minima while maintaining stability in the search process.
- **DAv3:** Builds upon DAv2 by adopting parallel tempering (replica exchange) as its primary search strategy. Multiple replicas at different temperatures are simulated concurrently, with periodic exchanges of configurations to improve global exploration. Furthermore, DAv3 integrates a hybrid framework that combines the DAU with a higher-level tabu-based global search running on multicore CPUs, enabling the solution of significantly larger fully connected QUBO instances, up to approximately 100,000 variables.
- **DAv3c:** A CPU-based software implementation of the DA framework that reproduces the optimization methodology of DAv3 without relying on specialized DAU hardware.

DAv2 and DAv3 are executed on specialized DAU hardware, enabling highly parallel.

In this work, we do not directly execute DAv2 or DAv3 on dedicated DA hardware. Instead, we use their reported benchmark results from the publicly available DAMB repository [14]. This ensures consistency with the original benchmark configuration and allows for a fair comparison against previously reported DA results.

Our experiments focus primarily on DAv3c, the CPU-based implementation of the DA framework. Since DAv3c operates entirely in software on conventional CPU architectures, it enables the underlying optimization methodology to be evaluated independently from the hardware acceleration provided by the DAU. DAv3c reproduces the core optimization strategies of DAv3, including parallel tempering, and adaptive global-search mechanisms. Although several of these components are related to techniques commonly used in classical metaheuristics, DAv3c integrates them within the DA optimization framework together with automatic parameter-management mechanisms designed for large-scale combinatorial optimization.

2.2.2 Classical Heuristic Baselines

To evaluate the performance of DAv3c, we compare it against a set of the best performing classical heuristics from the MQLib benchmark suite [15]. The benchmark consists of 37 algorithms spanning a range of metaheuristic paradigms, including local search, simulated annealing, tabu search, evolutionary methods, and hybrid approaches.

Rather than fixing a single heuristic across all instances, we follow the evaluation protocol of Shaglel et al. [5] and compare DAv3c against the best-performing heuristic within each size–density category (as defined in Section 3.2). This selection is based on prior large-scale benchmarking and ensures that each comparison is made against a strong and competitive baseline tailored to the structural characteristics of the problem instances. Throughout the remainder of this report, these category-specific selected baseline heuristics are collectively referred to as the *MQLib heuristics*.

This approach avoids bias toward any single heuristic and provides a robust reference point for assessing the relative performance of DAv3c.

Representative algorithms in this suite include `BURER2002`, which uses a continuous relaxation approach based on the Burer–Monteiro factorization, `PAL2004bMTS2`, based on Iterated Tabu Search, and `MERZ1999GLS`, which combines a Genetic Algorithm with local search [15], illustrating the diversity of optimization strategies considered.

Both DAv3c and the selected MQLib heuristics operate as software-based methods on conventional CPU architectures and are therefore evaluated without the hardware-level parallelism available to the DA, enabling a more direct comparison of algorithmic performance. However, both have been evaluated on different servers.

2.3 Benchmark Setup and Evaluation Protocol

We adopt the evaluation framework established in prior work [5], ensuring consistency in solver comparison and performance assessment. In particular, we follow the benchmark methodology used to evaluate DAv2 and DAv3 against the MQLib heuristics on different size–density instance categories described in Section 3.2.

2.3.1 Solver Scope and Comparisons

In the original benchmark, solver evaluations are subject to solver-specific constraints. In particular, no results are available for DAv2 on *xlarge* instances, while DAv3 is evaluated across a broader range of instance sizes.

In contrast, DAv3c does not impose such restrictions and can be applied across all instance categories. Therefore, in this work:

- DAv3c is compared against MQLib heuristics across all instance categories.
- DAv3c is compared against DAv2 and DAv3 only on the subset of instances where results for these solvers are available.

This setup ensures that all comparisons are conducted on overlapping instance sets while preserving consistency with the original benchmark protocol.

2.3.2 Baseline Time Limits

We adopt the instance-specific baseline time limits introduced in the benchmark study [15]. These limits are designed to reflect the computational difficulty of each instance, allocating more time to harder instances while avoiding excessive runtime for easier ones.

For DAv2, DAv3, and the MQLib heuristics, we use the same time limit configuration as defined in the Shaglel et al. [5] without modification.

DAv3c Time Limits

DAv3c allows explicit control over runtime within a range of $[1, 3600]$ seconds. To ensure compatibility with this constraint while maintaining consistency with the benchmark, we apply the following adjustments:

- Time limits below 1 second are set to 1 second.
- Time limits above 3600 seconds are capped at 3600 seconds.

After applying the time-limit filtering step (Section 3.1), a total of 590 instances have baseline time limits below 1 second. The effect of this adjustment is illustrated in Appendix A, where runtime behavior in the $0.25 <$ time limit < 1 regime is analyzed. In particular, DAv3 exhibits slight runtime overshooting due to internal iteration cycles, while DAv2 and the selected heuristics show comparatively smaller deviations. Notably, these instances predominantly belong to the *xsmall* and *small* categories and are excluded from the main pairwise comparisons (see Section 3.1).

At the upper end of the spectrum, only three instances have baseline time limits exceeding 3600 seconds. These correspond to *xlarge* instances, for which DAv2 results are not available. For these cases, the imposed cap leads to a reduced runtime budget for DAv3c relative to the original benchmark, where only DAv3 and the MQLib heuristics provide comparable results. A detailed comparison of runtime behavior and solution quality for these instances is provided in Appendix A.

2.3.3 Performance Evaluation

Each solver is evaluated based on the objective value obtained for a given instance. Following the evaluation protocol of Ref. [5], and to ensure direct comparability, we represent solver performance by the maximum objective value observed across multiple runs per instance.

To validate this choice, we analyze the variability across runs using CV and observe negligible dispersion for the vast majority of instances. This indicates that alternative statistics (e.g., mean or median) yield consistent conclusions and do not materially affect the comparative results (see Appendix B).

For pairwise comparisons, outcomes are classified into three categories: *win*, *loss*, or *tie*, based on the difference in objective values between solvers. A small tolerance of $\epsilon = 10^{-9}$ is applied to account for numerical precision, such that differences within ϵ are treated as ties.

Baseline Objective and Relative Gap Analysis

To analyze the impact of instance characteristics on solver performance, we construct a baseline objective value for each instance by taking the maximum objective value obtained across all evaluated solvers, including DAv3c. This baseline serves as a reference for the best-known solution quality within the experimental setting.

Using this baseline, we compute the relative gap for each solver as

$$\text{Gap} = \frac{f^* - f_{\text{solver}}}{f^*}, \tag{10}$$

where f^* denotes the baseline objective value, or the known optimal value for instances with available optima, and f_{solver} represents the objective value obtained by the corresponding solver. This normalized metric enables consistent comparison across heterogeneous instances with varying objective scales.

The resulting gap-based analysis is used to investigate solver performance across structural categories (Section 4.2), edge-weight concentration categories (Section 4.4), and on instances with known optimal solutions (Section 4.5).

To support reproducibility and facilitate further analysis, the source code, preprocessing scripts, analysis notebooks, and experimental configurations used in this study are publicly available.[1]. All DAv3c experiments were conducted on the Uranus server at TUHH equipped with an AMD EPYC 9275F processor (24 physical cores, 48 logical CPUs), 754 GiB RAM, running Red Hat Enterprise Linux 9.4 (Plow).

3 Dataset, Instance Selection, and Categorization

To ensure consistency and comparability with prior benchmarks of the DA (DAv2 and DAv3), this study adopts the same evaluation framework and dataset used in the original benchmark analysis. In particular, we utilize the MQLib repository [6], a comprehensive benchmark suite designed for evaluating heuristic algorithms for the Max-Cut problem. The dataset comprises 3,506 instances drawn from a diverse combination of real-world and synthetic sources, including established benchmark libraries

[1] https://github.com/sargun07/DAMB-DAv3c

Size	Density	Float	Integer	Total	MQLib Heuristics	Compared Against
XSmall	sparse	117	408	525	Merz	H (Appendix D)
	balanced	192	175	367	Burer	
	dense	65	117	182	Pal	
Small	sparse	37	142	179	Merz	H (Appendix D)
	balanced	33	8	41	Burer	
	dense	6	6	12	Pal	
Medium	sparse	108	171	279	Burer	H, DAv2, DAv3
	balanced	55	23	78	Burer	
	dense	27	23	50	Pal	
Large	sparse	100	71	171	Burer	H, DAv2, DAv3
	balanced	76	43	119	Burer	
	dense	23	18	41	Pal	
XLarge	sparse	47	34	81	Merz	H, DAv3
Total		**886**	**1248**	**2134**		

Table 31: Instance composition, selected MQLib heuristic, and solver comparison across size–density categories. Size categories are defined by the number of nodes n: XSmall $(20, 1024)$, Small $[1024, 2048)$, Medium $[2048, 4096)$, Large $[4096, 8192)$, and XLarge $[8192, \infty)$. Density categories are defined by graph density d: Sparse $[0, 0.1)$, Balanced $[0.1, 0.5)$, and Dense $[0.5, 1]$. Abbreviations: Burer = BURER2002, Pal = PAL2004bMTS2, Merz = MERZ1999GLS, H = MQLib Heuristics.

such as Gset, ORLib, SteinLib, Biq Mac, DIMACS, and TSPLib, as well as instances generated using tools such as Rudy, `NetworkX`, and the Culberson random graph generators. Consequently, the benchmark contains structurally heterogeneous graph families exhibiting substantial variation in graph topology, degree distributions, clustering behavior, community structure, and edge-weight distributions. This diversity enables a more comprehensive analysis of solver performance across different structural and edge-weight characteristics. Structural graph properties used throughout the analysis, including measures related to bipartiteness, triangles, clustering, and connectivity, were computed using the Python library `NetworkX` together with custom preprocessing scripts developed for this study.

3.1 Instance Filtering

We adopt the filtering strategy used in the benchmark study [5] to ensure consistency and comparability of results. Since the results for DAv3, DAv2, and the heuristics were not available for instances with a number of nodes ≤ 20, these 106 instances were removed. The final dataset therefore consists of 3400 instances.

Time-limit filtering

We exclude instances with baseline time limits below 0.25 seconds. As discussed in [5], such instances are typically computationally trivial and may lead to unreliable comparisons, as solvers can significantly overshoot the prescribed time limits due to infrequent internal time-limit checks. Removing these instances ensures that runtime constraints are meaningful and that observed performance differences reflect algorithmic behavior rather than artifacts of time-limit handling. After applying this filtering step, a total of 2125 instances remain for subsequent analysis.

Size-based filtering

In addition to time-limit filtering, we apply a size-based filtering step by excluding instances from the *xsmall* and *small* categories. This focuses the analysis on medium to extra-large instances, which are generally more challenging and provide greater discriminatory power when comparing solver performance. After applying this filtering step, 819 instances remain for the primary analyses.

This filtering is applied to the majority of analyses in this work, including pairwise solver comparisons and structural evaluations. However, it is *not* applied in the optimality analysis on bipartite instances (Section 4.5), where all available instances are retained to enable comparison against known optimal solutions.

For completeness, results for the excluded *xsmall* and *small* instances are provided in the appendix.

3.2 Size–Density Categorization

The filtered instances are partitioned into categories based on graph size and density, following the standard definitions in [5]. An overview of the resulting instance distribution is provided in Table 31. In total, this yields 13 size–density categories when including *xsmall* and *small* instances, and 7 categories when restricting attention to medium through xlarge instances.

This categorization forms the basis for all primary solver comparisons in this work. In particular, comparisons between DAv3c and the best-performing MQLib heuristics, as well as with DAv3, are conducted over the 7 medium-to-xlarge categories. For DAv2, the comparison is restricted to 6 categories, as it does not support xlarge instances. This ensures that all pairwise evaluations are performed on comparable and valid subsets of instances.

The primary analysis focuses on medium to xlarge instances, where the effect of enforced runtime constraints is less pronounced, providing a more consistent basis for comparison across solver configurations.

3.3 Structural Property Analysis

Structural properties of a graph can significantly influence the optimization landscape of the Max-Cut problem and, consequently, the behavior of different solvers [16]. To investigate these effects, we partition the instances into three categories based on bipartiteness and the presence of triangles: (i) **Bipartite** graphs, (ii) **Non-bipartite with triangles**, and (iii) **Non-bipartite triangle-free** graphs.

This categorization is motivated by the role of odd cycles in the Max-Cut problem and their connection to structural frustration, a well-known phenomenon in Ising and QUBO formulations [17, 18]. Bipartite graphs contain no odd cycles and, in the case of non-negative edge weights, admit an exact solution in which all edges can be simultaneously satisfied by the cut. In contrast, non-bipartite graphs necessarily contain odd cycles, introducing competing constraints that prevent all edges from being satisfied simultaneously.

Graphs containing triangles include the smallest possible odd cycles, although they may also contain longer odd and even cycles. In triangle-rich graphs, these short odd cycles may create tightly coupled local constraints, giving rise to localized frustration and rugged local energy landscapes. In contrast, triangle-free non-bipartite graphs contain no triangles but may still contain longer odd cycles (length ≥ 5) as well as even cycles. As a result, frustration is distributed across larger-scale cycle structures and competing edge interactions, leading to structurally different patterns of constraint interactions. These differences may affect how effectively optimization algorithms explore the solution space and escape local optima. In the following, we empirically evaluate how different solvers behave across these structural regimes.

Instance Filtering

In addition to the baseline filtering steps described in Section 3.1, further filtering is applied to isolate the effect of structural graph properties while minimizing density- and size-related confounding factors. Since triangle-free instances are naturally sparse, whereas instances containing triangles may span a broader range of graph densities, all structural categories are first restricted to sparse instances to obtain a more comparable setting.

After this filtering step, the remaining instances are concentrated in the *medium* and *large* size categories, with 279 *medium* and 171 *large* instances. The *xlarge* category is excluded because DAv2 results are unavailable for these instances due to hardware limitations. The final filtered dataset contains 450 instances across the three structural groups, and its distribution is summarized in Table 32.

Structure Category	**Medium**	**Large**	**Total**
Bipartite	55	27	82
Non-bipartite with triangles	145	103	248
Non-bipartite triangle-free	79	41	120
Total	**279**	**171**	**450**

Table 32: The table reports the number of instances in each structural category (bipartite, non-bipartite with triangles, and non-bipartite triangle-free) across the medium and large

Localized vs. Distributed Structural Frustration

To characterize the structural differences between the non-bipartite categories, we analyze five graph metrics: *Average Clustering* and *Transitivity* (local connectivity patterns), the *Cyclomatic Number* and *Bridge Fraction* (cycle structure and connectivity), and the *Negative Edge Fraction*. Definitions of these graph metrics are provided in Appendix E. As shown in Table 33, the two non-bipartite groups exhibit fundamentally different topological and weight-based characteristics.

Category	**Avg. Clust.**	**Trans.**	**Cyclo. #**	**Bridge Frac.**	**NE Frac.**
Triangle-Rich	0.106 (0.206)	0.043 (0.096)	10,001 (157,602)	0.001 (0.083)	0.000 (0.036)
Triangle-Free	0.000 (0.000)	0.000 (0.000)	6,737 (16,034)	0.000 (0.015)	0.000 (0.096)

Table 33: Structural metrics across non-bipartite graph categories. Avg. Clust. = Average Clustering Coefficient, Trans. = Transitivity, Cyclo. # = Cyclomatic Number, Bridge Frac. = Bridge Fraction, and NE Frac. = Negative Edge Fraction. For each structural category, the table reports median values of the corresponding graph properties computed across all instances, with mean values shown in parentheses.

The results suggest a distinction between *localized* and *distributed* forms of structural frustration.

Triangle-Rich (Localized Structural Dependencies): These instances exhibit substantial local clustering, with a median clustering coefficient of approximately 0.106, indicating the presence of tightly connected local neighborhoods and short-cycle interactions. The comparatively higher bridge fraction (mean $\approx$ 0.083, median $\approx$ 0.001) further suggests a partially modular topology in which locally dense regions are connected through relatively sparse inter-community links. Although the median negative edge fraction remains zero, the overall negative-edge ratio is comparatively lower than in the triangle-free category (mean $\approx$ 0.036), indicating that structural complexity in these instances is driven primarily by localized connectivity patterns and triangular interactions rather than competing signed-edge constraints.

Triangle-Free (Distributed Structural Dependencies): These instances exhibit no local clustering, as reflected by zero clustering coefficient and transitivity across all instances. Despite the absence of triangles, the graphs remain highly cyclic, with a median cyclomatic number of approximately 6,737, indicating the presence of numerous longer-cycle interactions. In contrast to the triangle-rich category, the bridge fraction remains close to zero (mean $\approx$ 0.015), suggesting a more globally interconnected structure with fewer separable local regions. Furthermore, the triangle-free category exhibits a comparatively higher negative edge fraction (mean $\approx$ 0.096), indicating that competing signed-edge constraints are more widely distributed throughout the graph. Together, these properties suggest that structural dependencies in triangle-free instances emerge through larger-scale cyclic interactions rather than localized clustered structures.

3.4 Edge-Weight Representation

In addition to structural graph characteristics, we also analyze the role of edge-weight representation in solver performance. This consideration is particularly relevant for the DA family of solvers, as non-integer edge weights are internally scaled into integer-valued representations prior to optimization. Since this preprocessing step is only required for non-integer instances, we investigate whether the distinction between integer and non-integer edge weights is associated with observable differences in solver performance.

Among the filtered instances, 46.76% (383) contain exclusively integer-valued edge weights, while 53.24% (436) contain non-integer edge weights. Notably, the non-integer category is dominated by floating-point weighted instances, with 430 instances containing exclusively floating-point edge weights and only 6 instances containing a mixture of integer and floating-point weights. The distribution of integer and non-integer instances is also closely related to the heterogeneous construction of the MQLib benchmark library [6]. Classical benchmark families such as the Beasley, GKA, and P3-7 instances contain exclusively integer-valued edge weights, whereas graph families generated using random graph generators, such as the Culberson instances, predominantly contain floating-point edge weights. In addition, the benchmark construction samples edge weights from multiple probability distributions, including continuous distributions [6], which naturally produces a larger number of distinct edge-weight values.

Figure 31 illustrates the relationship between edge-weight representation and unique-weight diversity. Integer-valued instances are predominantly associated with lower numbers of distinct edge weights, whereas non-integer instances exhibit substantially larger unique-weight counts.

To examine the impact of edge-weight representation on solver behavior, we analyze DA performance separately on integer and non-integer instances across different size-density categories and compare the resulting win, tie, and loss distributions against the selected MQLib heuristics.

3.5 Dominant Edge-Weight Concentration

To further investigate the influence of edge-weight structure on optimization behavior, we analyze solver performance across varying degrees of dominant edge-weight concentration. Repeated edge weights increase the likelihood of identical or near-identical objective contributions, potentially producing more degenerate optimization landscapes with many equivalent local configurations. Related effects of edge-weight structure on the Max-Cut objective landscape were reported by Shaydulin et al. [16], who observed that weighted Max-Cut instances especially those with broad variance in weights exhibits a wide spread of suboptimal local maxima as compared to the unweighted instances. Since optimization strategies may respond differently to such landscape characteristics, this analysis compares the behavior of the evaluated solvers under different levels of edge-weight repetition.

To ensure comparability across all four solvers, the same filtering strategy used in the previous analyses is applied. Instances with a runtime limit factor below 0.25 are

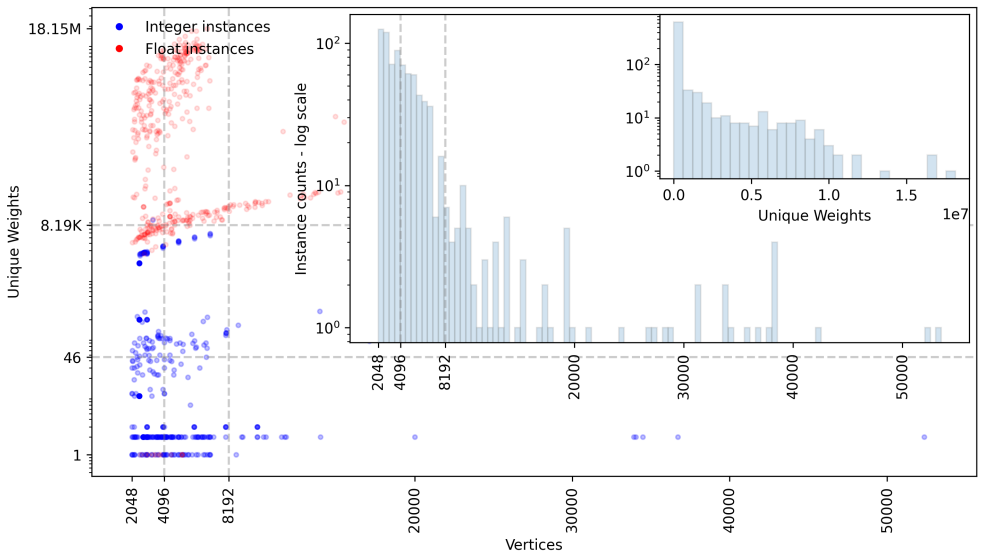


Fig. 31: Relationship between edge-weight representation and the number of unique edge weights across benchmark instances. Non-integer instances generally exhibit substantially higher weight diversity than integer-weighted instances.

excluded, together with *xsmall*, *small*, and *xlarge* instances. The exclusion of *xlarge* instances is necessary because DAv2 results are unavailable for these instances under the evaluated experimental setting. After filtering, the resulting subset contains 738 instances, consisting of 389 floating-point-weighted and 349 integer-weighted instances.

As discussed in Section 3.4, non-integer and integer-weighted benchmark instances originate from different graph-generation and edge-weight construction methodologies. Consequently, they are analyzed separately to reduce potential confounding effects arising from benchmark construction rather than edge-weight concentration itself. Examination of the non-integer-weighted instances shows that more than 95% of edge weights are unique in almost all cases, indicating highly heterogeneous weight distributions consistent with their generation process. Therefore, the dominant edge-weight concentration analysis focuses primarily on the integer-weighted instances.

For integer-weighted instances, dominant edge-weight concentration is characterized using the n_{90} metric, defined as the minimum number of unique edge-weight values required to account for 90% of all edge occurrences within a graph. Smaller n_{90} values indicate stronger concentration around a limited set of repeated edge weights, whereas larger values correspond to increasingly heterogeneous weight distributions.

Based on this metric, the integer-weighted instances are divided into four categories, summarized in Table 34. Instances containing only a single unique edge-weight value are classified as *uniform*. Instances with $n_{90} \leq 5$ are categorized as *concentrated*, representing graphs dominated by a very small number of repeated edge weights. Instances with $5 < n_{90} \leq 50$ are classified as *transitional*, reflecting intermediate

Category	Condition	Count
Uniform	Unique weights = 1	49
Concentrated	$n_{90} \in [1, 5]$	123
Transitional	$n_{90} \in (5, 50]$	73
Heterogeneous	$n_{90} > 50$	104

Table 34: Categorization of integer-weighted instances based on dominant edge-weight concentration. The metric n_{90} denotes the minimum number of unique edge-weight values required to account for 90% of all edge occurrences within a graph. "Unique weights" refers to the total number of distinct edge-weight values present in the instance.

levels of edge-weight diversity. Finally, instances with $n_{90} > 50$ are categorized as *heterogeneous*, indicating substantially more diverse edge-weight distributions.

The resulting distribution of integer-weighted instances shows that concentrated instances form the largest category ($n = 123$), followed by heterogeneous instances ($n = 104$), transitional instances ($n = 73$), and uniform instances ($n = 49$). Overall, this distribution suggests that repeated edge-weight structures are common throughout the benchmark collection, while a substantial subset of instances still exhibits highly diverse edge-weight configurations. Additional details regarding the distribution of n_{90} values are provided in Appendix C.

3.6 Optimality Analysis on Bipartite Instances with Positive Weights

Unlike the pairwise solver comparisons presented in earlier sections, this analysis does not apply the size-based filtering described in Section 3.1. Since the objective here is to evaluate solver accuracy with respect to a known optimum, smaller instances remain informative despite being comparatively easy. Consequently, only the baseline time-limit filtering from Section 3.1 is retained.

For bipartite graphs with non-negative edge weights, the Max-Cut objective admits a direct optimal solution: all edges can be placed across the partition, and thus the Max-Cut value equals the sum of all edge weights. This property allows the exact optimum to be computed efficiently and enables a direct evaluation of the success rate of each solver. In contrast, bipartite instances containing negative edge weights do not admit this trivial characterization, since cutting all edges is no longer necessarily optimal.

Based on this observation, we isolate all bipartite instances with zero negative edge weights and evaluate solver performance against the true optimum. The resulting subset consists of 146 instances, the majority of which are integer-weighted (130 integer

Solver	#Instances
DAv2	136
DAv3	146
DAv3c	146
Heuristic	146

(a) Instance distribution across solver configurations

Size Category	#Instances
xsmall	40
small	30
medium	42
large	24
xlarge	10

(b) Size distribution of the filtered subset

Table 35: Summary of the filtered bipartite instances with positive edge weights. The final subset contains 146 instances.

vs. 16 non-integer instances). The size distribution of these instances is summarized in Table 35b.

All selected instances fall within the sparse density regime. The number of evaluated instances per solver is summarized in Table 35a. Since DAv2 does not provide results for *xlarge* instances under the considered experimental setup, the 10 *xlarge* instances are excluded from the main solver comparison to ensure consistency across all evaluated methods. Consequently, the primary comparison is performed on the remaining 136 instances for which results are available from all solvers. The results for the excluded *xlarge* instances are reported separately.

This controlled setting enables a precise assessment of solver optimality, as any deviation from the optimal value directly reflects a failure to reach the global optimum rather than approximation quality.

4 Results

4.1 Performance Across Size–Density Categories

4.1.1 DAv3c vs. MQLib Heuristics

We evaluate the performance of DAv3c against the best-performing MQLib heuristic selected per size–density category. Figure 41 and Figure 42 summarize the comparative performance in terms of relative solver outcomes (wins, ties, and losses) and cut-value ratios.

Results for *small* and *xsmall* instances are omitted from this section and are instead presented in Appendix D. These instances are generally considered computationally easy and exhibit a high proportion of ties, making them less informative for distinguishing solver performance at scale.

Across all instances, DAv3c achieves 360 wins (43.96%), 373 losses (45.54%), and 86 ties (10.50%), indicating an overall competitive but slightly disadvantageous performance relative to the selected MQLib heuristics.

Performance varies systematically with graph density. DAv3c performs strongly in sparse regimes at moderate scales, achieving a clear advantage in *medium–sparse*

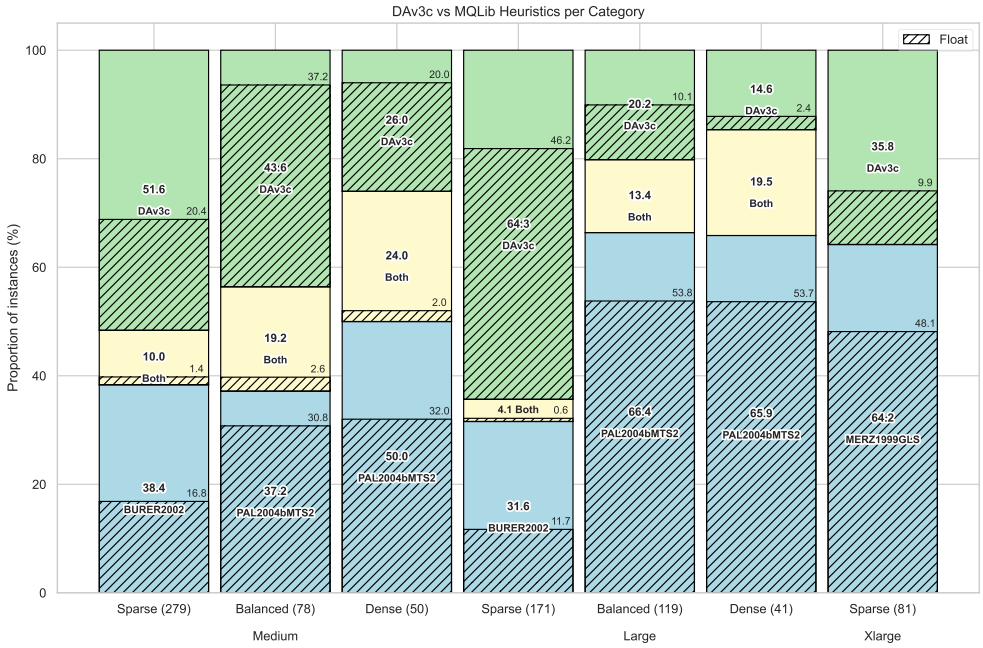


Fig. 41: Performance comparison between DAv3c and the category-selected MQLib heuristics across size–density groups. Green, yellow, and blue denote DAv3c wins, ties, and heuristic wins, respectively. The percentage labels inside each segment indicate the proportion of instances belonging to that outcome within a category, summing to 100% per category. Hatched regions correspond to non-integer-weighted instances, and the associated annotations indicate their proportion relative to the total number of instances in the category.

(144 wins vs. 107 losses) and *large–sparse* (110 wins vs. 54 losses). However, this advantage does not persist uniformly with increasing size, as *xlarge–sparse* instances show a reversal (29 wins vs. 52 losses).

In contrast, performance degrades consistently as density increases. In *balanced* and *dense* categories, the selected heuristics increasingly outperform DAv3c, most notably in *large–balanced* (79 losses vs. 24 wins) and *large–dense* instances.

These results indicate that density, rather than size alone, is the primary factor governing relative performance, while scaling effects are non-uniform.

The distribution of cut ratios (Figure 42) is strongly concentrated around 1. Approximately 62.1% of all instances fall within $[0.998, 1.002]$, including 10.50% exact matches, indicating that most performance differences are small in magnitude.

A clear distinction emerges between edge-weight types. Non-integer instances contribute to both wins and losses, whereas integer-weight instances are concentrated near the cut ratio of 1, accounting for most ties. This suggests that discrete weight structures lead to higher frequencies of equivalent solutions.

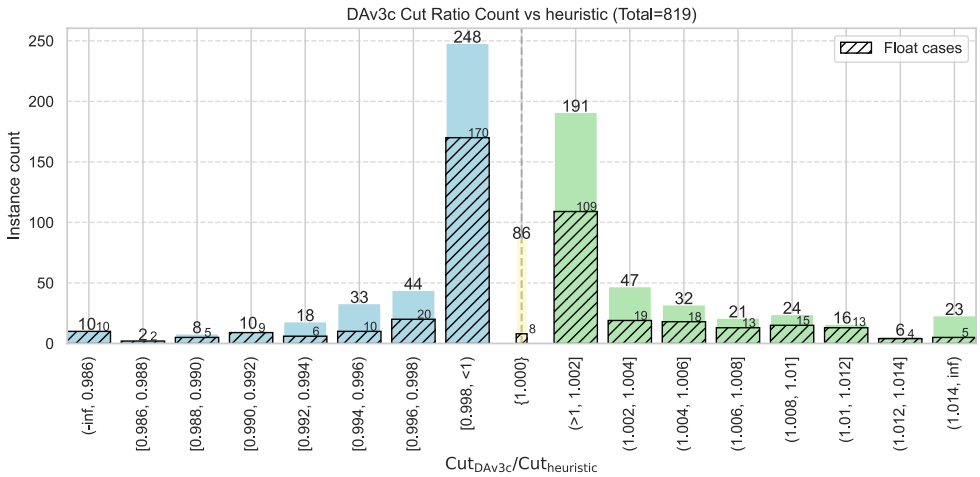


Fig. 42: Distribution of cut-value ratios across all considered instances. The ratio is defined as $\mathrm{Cut}_{\mathrm{DAv3c}}/\mathrm{Cut}_{\mathrm{heuristic}}$. Values close to 1 indicate near-identical solution quality, while deviations above or below 1 reflect relative improvements or degradations in DAv3c performance compared to the selected heuristic.

Statistical Validation

To assess the robustness of these observations, we perform paired Wilcoxon signed-rank tests [19, 20] across categories explain in Appendix F.

Most categories exhibit statistically significant differences, including *large–sparse* ($p = 1.86\times10^{-9}$, $r = 0.468$) and *xlarge–sparse* ($p = 1.96\times10^{-6}$, $r = 0.529$), confirming that the observed performance patterns are not due to random variation.

In contrast, *medium–balanced* shows no significant difference ($p = 0.909$, $r = 0.014$), indicating near-equivalent performance in this regime.

Overall, these results support the observation that performance differences across size–density categories are statistically robust, with graph density emerging as the dominant organizing factor of solver behavior.

4.1.2 DAv3c vs. DAv3

Figure 43 presents the performance comparison between DAv3c and DAv3 across size–density categories.

DAv3c exhibits a clear advantage only in *xlarge–sparse*, where it achieves 61 wins compared to 20 losses. In all other categories, DAv3 consistently outperforms DAv3c, often by substantial margins, particularly in dense and medium-sized regimes.

Notably, DAv3c maintains its advantage in the *xlarge–sparse* regime even under unfavorable runtime conditions. In the three instances where DAv3c is disadvantaged by the imposed maximum runtime limit (Appendix A), it still achieves superior cut values in all cases. A likely explanation is that DAv3 applies a decomposition-based strategy for the x-large instances, partitioning the graph into smaller subproblems

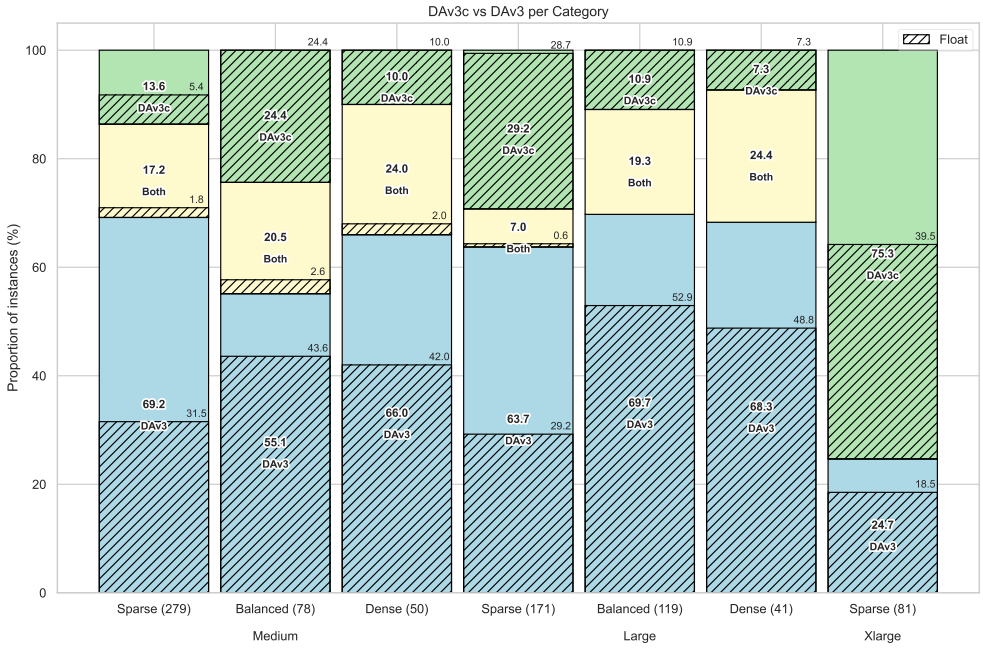


Fig. 43: Performance comparison between DAv3c and DAv3 across size–density groups using the same visualization scheme as Figure 41.

before combining the resulting solutions, whereas DAv3c operates directly on the full instance.

The corresponding cut-ratio distribution (Appendix G) is left-skewed, reflecting the overall dominance of DAv3. Nevertheless, most instances remain concentrated close to cut ratio of 1, indicating that the majority of performance differences are relatively small in magnitude despite the consistent win/loss trends.

Statistical testing confirms these observations. All categories show statistically significant differences (Appendix F), with large effect sizes in dense regimes (e.g., *large–dense*, $r = 0.764$), indicating that DAv3 provides consistently stronger performance across the benchmark except for xlarge instances.

4.1.3 DAv3c vs. DAv2

Figure 44 presents the comparison between DAv3c and DAv2.

Across all instances, DAv3c records 207 wins (28.05%), 455 losses (61.66%), and 76 ties (10.30%), indicating a clear disadvantage relative to DAv2.

Performance varies across categories. DAv2 consistently outperforms DAv3c in dense regimes, particularly in *medium–dense* (10 wins vs. 34 losses) and *large–dense* (9 wins vs. 28 losses). In sparse regimes, the gap becomes less pronounced. For example, in *large–sparse*, DAv3c records 62 wins compared to 99 losses; however, the corresponding statistical test shows no significant difference for this category, suggesting that the observed imbalance is less robust than in the dense regimes.

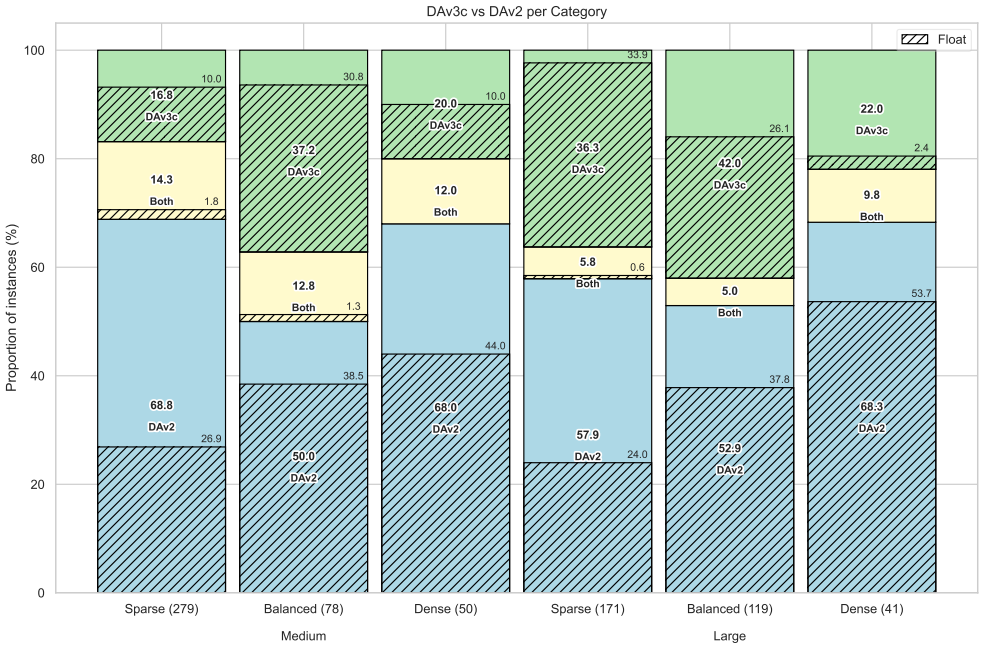


Fig. 44: Performance comparison between DAv3c and DAv2 across size–density groups using the same visualization scheme as Figure 41.

The cut-ratio distribution (Appendix H1) is left-skewed, reflecting the overall advantage of DAv2. Nevertheless, a large proportion of instances remain concentrated close to the cut ratio of 1, particularly within the interval $[0.998, 1)$, indicating that many performance differences are relatively small in magnitude.

Statistical tests further confirm these trends. Significant differences are observed in several categories, particularly *medium–dense* ($p = 6.48 \times 10^{-5}$, $r = 0.575$), while sparse categories generally exhibit weaker statistical separation.

4.2 Solver Performance Across Structural Categories

Building on the baseline objective defined in Section 2.3.3, we evaluate solver performance using the *success rate*, defined as the percentage of instances for which a solver attains the baseline (i.e., best-known) objective value. Table 41 summarizes the success rate (in %) of each solver across the three structural categories, restricted to sparse instances.

Across all structural categories, DAv3 achieves the highest success rate on bipartite and triangle-rich non-bipartite instances, reaching 97.56% and 77.42%, respectively. On triangle-free non-bipartite instances, DAv2 attains the highest success rate at 63.33%, exceeding the performance of DAv3 in this category.

DAv3c achieves moderate success on bipartite instances (43.90%), but its performance decreases substantially across the non-bipartite categories, reaching 24.19% on triangle-rich graphs and only 8.33% on triangle-free graphs.

Solver	Bipartite (N=82)	NB + Triangles (N=248)	NB Triangle-Free (N=120)
DAv3c	43.90 [33.67,54.68]	24.19 [19.28,29.89]	8.33 [4.59,14.66]
DAv3	**97.56 [91.54,99.33]**	**77.42 [71.82,82.18]**	51.67 [42.81,60.42]
DAv2	67.07 [56.34,76.28]	33.06 [27.51,39.14]	**63.33 [54.42,71.42]**
Heuristic	34.15 [24.80,44.91]	0.40 [0.07,2.25]	5.83 [2.85,11.55]

Table 41: Success rate (%) with 95% Wilson confidence intervals [21] across sparse structural graph classes. Success rate is defined as the proportion of instances for which a solver achieves the best cut value observed among all compared solvers. Values are reported as percentages with confidence intervals in brackets. Column headers indicate the number of instances (N) in each category. NB denotes non-bipartite graphs. The best performance within each structural category is highlighted in bold.

The heuristic baseline shows comparatively limited success across all categories, achieving 34.15% on bipartite instances, 5.83% on triangle-free non-bipartite graphs, and only 0.40% success on triangle-rich non-bipartite graphs.

4.3 Edge-Weight Representation Analysis

To investigate the effect of edge-weight representation, we analyze solver performance separately for integer-weighted and non-integer-weighted instances across all evaluated graphs, as shown in Figure 45. Since DAv2 does not provide results for *xlarge* instances under the considered experimental setup, the aggregated solver comparison excludes the *xlarge* category to ensure consistency across all evaluated solvers. The corresponding *xlarge* results for DAv3 and DAv3c are reported separately in Table 42.

Statistical significance of the observed performance differences is evaluated using the Wilcoxon signed-rank test, with detailed results reported in Appendix I. The aggregated comparisons for DAv2 and DAv3 are statistically significant for both integer and non-integer instances, whereas the corresponding DAv3c comparisons are not statistically significant at $\alpha = 0.05$. Consequently, the DAv3c trends discussed in this section should be interpreted as descriptive observations rather than statistically validated effects.

Additional solver-wise comparisons across individual size–density categories are provided in Appendix J.

Despite the DA being aninteger machine, both DAv2 and DAv3 remain competitive on non-integer instances.

4.4 Solver Performance Across Edge-Weight Concentration Categories

Table 43 summarizes the percentage of instances for which each solver achieves the best baseline objective value across the different edge-weight concentration categories. Overall, DAv3 consistently achieves the highest performance across all categories, while DAv2 represents the second strongest performer. In contrast, DAv3c and the

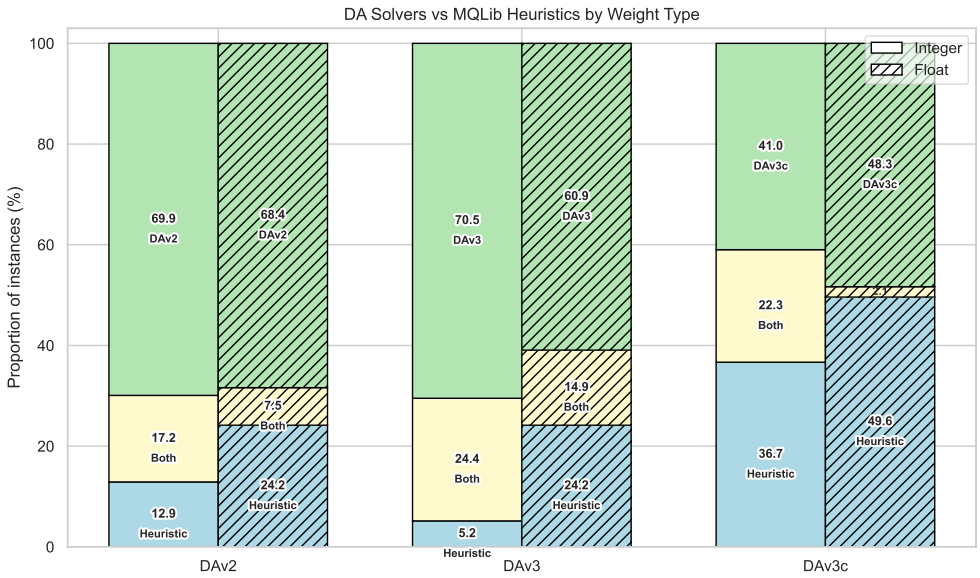


Fig. 45: Aggregate performance comparison of DAv2, DAv3, and DAv3c against the category-specific MQLib heuristic baselines for integer-weighted and non-integer-weighted instances.

Solver	Weight Type	Loss	Tie	Win
DAv3	Float	43	0	4
	Integer	13	0	21
DAv3c	Float	39	0	8
	Integer	13	0	21

Table 42: Win–tie–loss comparison by edge-weight representation for *xlarge* instances.

heuristic baseline exhibit substantially lower hit-best rates throughout the evaluated categories.

A clear performance trend is observed for DAv3 as edge-weight diversity increases. DAv3 achieves the best baseline objective value for all uniform instances (100%), while its performance gradually decreases for concentrated (97.56%), transitional (84.93%), and heterogeneous (75.96%) instances. A similar, although less consistent, trend is observed for DAv2, whose performance decreases from concentrated (51.22%) to transitional (30.14%) instances before partially recovering for heterogeneous instances (51.92%). DAv3c and the heuristic baseline also exhibit reduced performance as edge-weight diversity increases, although their overall hit-best rates remain substantially below those of DAv3 and DAv2.

Solver	Uniform (n=49)	Concentrated (n=123)	Transitional (n=73)	Heterogeneous (n=104)
DAv2	36.73 [24.67, 50.73]	51.22 [42.48, 59.88]	30.14 [20.82, 41.44]	51.92 [42.42, 61.28]
DAv3	**100.00 [92.73, 100.00]**	**97.56 [93.07, 99.17]**	**84.93 [75.00, 91.37]**	**75.96 [66.92, 83.15]**
DAv3c	59.18 [45.25, 71.78]	30.89 [23.41, 39.53]	24.66 [16.20, 35.64]	31.73 [23.57, 41.19]
Heuristic	40.82 [28.22, 54.75]	25.20 [18.36, 33.55]	15.07 [8.63, 25.00]	23.08 [16.03, 32.05]

Table 43: Success rate (%) with 95% Wilson confidence intervals across edge-weight concentration categories using the same definition of success rate as in Table 41. The best performance within each category is highlighted in bold.

The confidence intervals further indicate that the dominant performance advantage of DAv3 remains stable across all edge-weight concentration categories. In particular, the uniform category exhibits a lower confidence bound above 92%, indicating highly consistent performance despite the relatively smaller number of instances in this subset.

Since DAv3 achieved perfect performance on the filtered set of uniform instances, we additionally examined the complete collection of uniform benchmark instances, including the previously excluded *xsmall* and *small* categories. Across all 173 uniform instances, DAv3 continued to achieve the best baseline objective value in every case (100%), suggesting that its strong performance on highly concentrated edge-weight structures remains stable across graph sizes. One possible explanation for this behavior is the use of tabu-search-based mechanisms within the DA framework. Tabu Search employs adaptive memory structures to discourage revisiting recently explored configurations and to guide the search beyond local optimality [22, 23]. Such mechanisms may be particularly beneficial in highly degenerate optimization landscapes containing many equivalent or near-equivalent solutions induced by repeated edge weights.

To further investigate the relative behavior of the two strongest-performing solvers, we additionally perform a pairwise comparison between DAv3 and DAv2 on the filtered benchmark subset. The results, summarized in Table 44, show that DAv3 either outperforms or ties with DAv2 for the vast majority of uniform and concentrated instances, with DAv2 wins being nearly negligible in these highly repeated edge-weight regimes. As edge-weight diversity increases, however, the relative performance gap between the two solvers decreases, and the DAv2 win rate correspondingly increases, particularly for heterogeneous instances. These trends suggest that DAv3 maintains a stronger advantage in optimization landscapes dominated by repeated edge weights, whereas DAv2 becomes comparatively more competitive as the edge-weight distributions become increasingly diverse.

4.5 Solver Accuracy and Gap Distribution

Having established a subset of bipartite instances with known optimal solutions (Section 3.6), we evaluate solver performance in terms of both optimality frequency and deviation from the optimum. As these instances are bipartite, sparse, and contain only non-negative edge weights, they represent a structurally simple class of Max-Cut instances. The results should therefore be interpreted in the context of this comparatively easy setting.

Category	DAv3	Tie	DAv2
Uniform	63.27	36.73	0.00
Concentrated	48.78	48.78	2.44
Transitional	68.49	23.29	8.22
Heterogeneous	48.08	27.88	24.04

Table 44: Pairwise comparison between DAv3 and DAv2 across edge-weight concentration categories (%).

Optimality Rate

Figure 46 reports the percentage of instances for which each solver attains the optimal solution, corresponding to a zero gap. To ensure a consistent comparison across solvers, both the optimality-rate plot and the gap-distribution analysis are restricted to the common subset of 136 instances, excluding the 10 *xlarge* instances for which DAv2 results are not available.

On this common subset, DAv3 achieves the highest optimality rate, solving 96.3% of instances to optimality, followed by DAv2 with 91.9%. DAv3c and the heuristic baseline achieve lower but similar optimality rates, with 73.5% and 74.3%, respectively.

For completeness, the excluded *xlarge* instances are evaluated separately for the solvers with available results. On these 10 instances, DAv3 and the heuristic baseline fail to reach optimality on all 10 cases, whereas DAv3c fails on 6 cases.

Gap Distribution

While optimality rates indicate how often solvers reach the optimum, they do not capture the magnitude of suboptimality. To provide a more detailed view, Figure 47 presents the empirical cumulative distribution function (CDF) of the relative gap to the optimal value, again computed on the common subset of 136 instances.

The CDF results confirm the ranking observed in the optimality-rate comparison. DAv3 achieves the strongest overall performance, solving 96.32% of instances within a gap tolerance of 10^{-6} and 97.79% within a gap of 10^{-3}. DAv2 follows closely, reaching 91.91% within a gap tolerance of 10^{-6} and 92.65% within a gap of 10^{-3}.

DAv3c and the heuristic baseline exhibit nearly identical behavior across the smallest gap thresholds, with both achieving 74.26% within a gap tolerance of 10^{-6}. At a 1% gap threshold, the proportion of near-optimal solutions increases substantially for both solvers, reaching 91.91% for DAv3c and 91.18% for the heuristic baseline. All solvers attain 100% within a gap of 5×10^{-2}, with the exception of DAv3, which already reaches 99.26% within a 1% gap threshold.

5 Discussion

The results of this study provide several insights into the performance characteristics of DAv3c relative to both specialized DA hardware implementations (DAv2 and DAv3) and classical heuristics from MQLib.

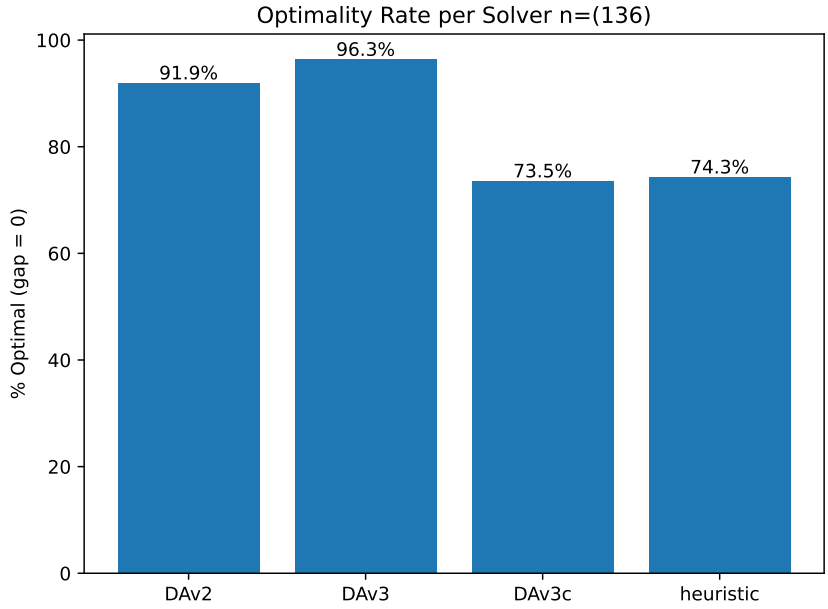


Fig. 46: Optimality rate (%) of each solver on the common subset of 136 bipartite instances with non-negative edge weights. Optimality rate is defined as the proportion of instances for which the solver attains the true optimal cut value. The 10 *xlarge* instances are excluded because DAv2 results are not available for this size category.

First, the comparison with DAv2 and DAv3 highlights the substantial impact of hardware acceleration. Across most instance categories, the hardware-based solvers consistently outperform DAv3c. Since DAv3c incorporates the same algorithmic principles as the DAv3, the observed performance gap is likely attributable to the architectural advantages of the DAU, particularly its large-scale parallelism and specialized optimization hardware. These observations support the hypothesis that a significant portion of the performance advantage reported in prior DA benchmarks arises from hardware acceleration rather than purely from algorithmic improvements.

Second, when compared against the category-specific best-performing MQLib heuristics, DAv3c demonstrates competitive but overall slightly weaker performance. In particular, DAv3c performs comparatively well on sparse graph categories, while its relative performance decreases as graph density increases. This suggests that DA-style optimization dynamics remain effective for certain classes of optimization landscapes, whereas classical heuristics continue to perform strongly on denser and more structured instances where highly specialized local-search procedures are particularly effective.

Third, structural graph properties, particularly those related to odd-cycle structure, appear to influence solver behavior. All solvers achieve comparatively strong performance on bipartite instances, where odd cycles are absent and the resulting optimization landscape is structurally less constrained. However, more pronounced

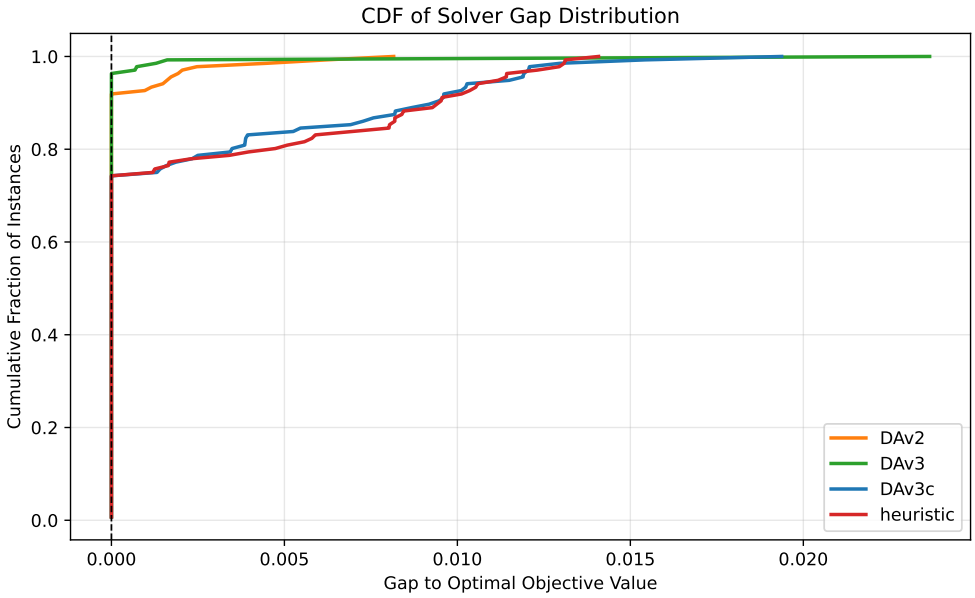


Fig. 47: Empirical cumulative distribution of the relative gap to the true optimal value for bipartite instances with non-negative edge weights. Curves closer to the top-left indicate better solver performance, corresponding to smaller relative gaps across a larger proportion of instances.

differences emerge on non-bipartite graphs. In particular, DAv3c and DAv3 perform comparatively better on instances containing triangular structures, whereas DAv2 exhibit relatively stronger performance on triangle-free instances characterized by longer-cycle interactions.

Rather than attributing these differences to specific internal algorithmic mechanisms, we interpret these observations more conservatively as evidence that solver performance is influenced by odd-cycle structure. In this work, odd-cycle characteristics are approximated using bipartiteness and triangle-related features, since explicit enumeration of longer cycles is computationally impractical for many large-scale instances. These structural proxies therefore provide only a partial characterization of the underlying graph structure, and the observed trends should be interpreted with caution. A more detailed investigation of cycle structures, particularly involving longer odd cycles, may provide additional insight into solver behavior and represents an interesting direction for future work.

Fourth, the analysis of edge-weight characteristics reveals an important interaction between edge-weight representation, unique-weight diversity, and solver outcomes. All DA variants experience comparatively more losses on non-integer instances, with DAv3c exhibiting the most pronounced degradation relative to DAv2 and DAv3.

On integer-weighted instances, DAv3 achieves 100% performance on uniformly weighted instances, while its performance gradually decreases as the edge-weight distribution becomes more heterogeneous. In contrast, DAv2 exhibits comparatively improved performance on instances with increasingly heterogeneous edge-weight distributions.

Finally, the optimality analysis on easy bipartite instances demonstrates that the hardware-based DA variants achieve near-perfect optimality rates, significantly outperforming both DAv3c and the selected MQLib heuristics. These results suggest that even in comparatively simple optimization settings, the hardware-based implementations are substantially more reliable in consistently reaching optimal solutions within the allocated runtime limits.

Overall, the findings of this study indicate that solver performance is not uniform across instance categories, but instead depends on a combination of hardware capabilities, optimization strategies, structural graph properties, and edge-weight characteristics.

6 Conclusion

This work presented a comprehensive evaluation of DAv3c, a CPU-based implementation of the DA framework, on a large and structurally diverse collection of Max-Cut benchmark instances. By comparing DAv3c against both hardware-based DA variants (DAv2 and DAv3) and state-of-the-art classical heuristics from MQLib, this study aimed to better separate the contribution of algorithmic design from the impact of specialized hardware acceleration.

The results show that although DAv3c incorporates several of the key optimization mechanisms used in the DAv3 framework, including stochastic annealing dynamics and replica-based exploration strategies, it does not achieve the same level of performance as the hardware-based implementations. This suggests that the strong performance reported in prior DA benchmarks depends not only on the underlying optimization strategy, but also substantially on the specialized architecture and large-scale parallelism provided by the DAU.

At the same time, DAv3c remains competitive with classical heuristics across several instance categories, particularly in sparse graph regimes. However, its relative performance decreases as graph density increases, where classical heuristics continue to demonstrate strong effectiveness. These observations indicate that solver performance is highly dependent on instance characteristics and structural properties rather than being uniformly transferable across all problem classes.

Additional analyses further revealed that both edge-weight representation and graph structure influence solver behavior. Integer-weighted instances consistently produced substantially larger numbers of ties across all solver variants, whereas non-integer instances resulted in more decisive outcomes with clearer win–loss separations. Furthermore, graph properties related to odd-cycle structure also appeared to affect solver competitiveness, with different solvers exhibiting relative strengths on different classes of instances. The analysis of dominant edge-weight concentration additionally showed that solver performance varies with edge-weight diversity, with

DAv3 performing particularly strongly on uniformly weighted and highly concentrated integer-weighted instances, while DAv2 becomes comparatively more competitive as edge-weight distributions become increasingly heterogeneous.

Overall, the findings of this study demonstrate that aggregate benchmark performance alone may obscure important differences between optimization methods and instance classes. Consequently, detailed instance-aware benchmarking is essential for understanding the interaction between optimization strategies, hardware acceleration, and structural graph properties when evaluating both classical and hardware-accelerated optimization approaches.

Several directions remain open for future work. First, the influence of odd-cycle structure could be investigated more rigorously through improved methods for identifying and characterizing longer odd cycles in large-scale graphs. Since the current study relies primarily on bipartiteness and triangle-based approximations, a more detailed cycle-level analysis may provide deeper insight into solver behavior.

Second, future work could investigate whether similar edge-weight concentration effects also emerge for non-intger-weighted instances by constructing controlled benchmark datasets containing repeated floating-point edge weights. Such experiments would help determine whether the trends observed for integer-weighted instances generalize more broadly to repeated edge-weight structures independent of the numerical representation itself.

Declaration of AI Usage

AI-based tools (ChatGPT) were used in a limited capacity to assist with paraphrasing, language refinement, and LaTeX formatting. The tool was also used to cross-check calculations and verify intermediate analytical results.

All substantive aspects of the work, including problem formulation, experimental design, implementation, data analysis, and conclusions, were developed independently by the authors. The authors take full responsibility for the content of this work.

Code Availability

The source code, preprocessing scripts, analysis notebooks, and experimental configurations used in this study are publicly available at:

https://github.com/sargun07/DAMB-DAv3c

Appendix A Runtime Behavior at Extreme Time Limits TL

To further examine solver behavior under extreme runtime conditions, we analyze two subsets of instances based on their baseline time limits: (i) instances with short time limits ($0.25 < \text{TL} < 1$ seconds), and (ii) instances with very large time limits ($\text{TL} > 3600$ seconds).

Short Time Limits *(*$0.25 <$ ***TL*** < 1*)*

Figure A1 illustrates the runtime distribution across solvers for instances with short time limits. In this regime, runtime measurements are affected by solver-specific implementation details, such as infrequent time-limit checks and minimum runtime constraints. In particular, DAv3c enforces a minimum runtime of 1 second, leading to systematic overshooting relative to the prescribed limits. DAv3 also exhibits overshooting behavior due to its minimum runtime constraint and the requirement to complete internal optimization cycles before termination.

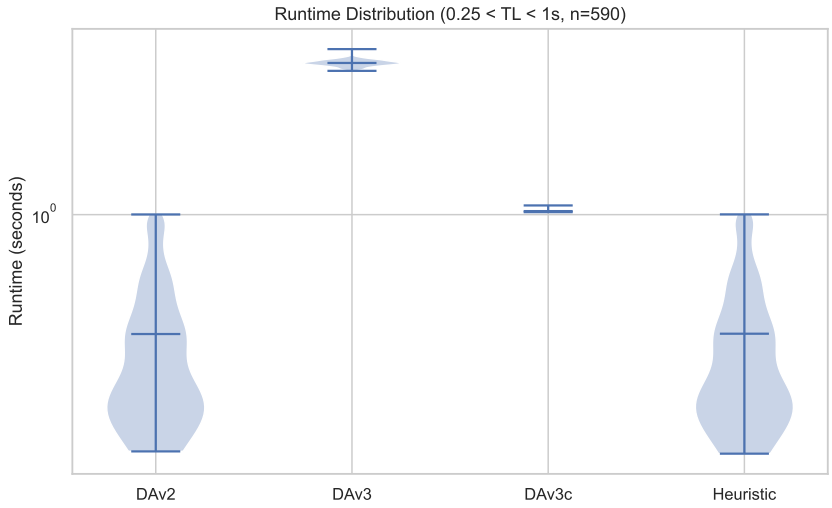


Fig. A1: Violin plot of solver runtime distributions for instances with baseline time limits (TL) satisfying $0.25 <$ TL < 1 seconds. Wider regions indicate a higher concentration of instances at the corresponding runtime values.

Long Time Limits *(****TL*** > 3600*)*

There are three instances for which the time limits exceeds 3600 seconds. For these cases, DAv3c is evaluated under a capped runtime of 3600 seconds, while the original benchmark allows longer runtimes for other solvers.

Table A1 reports solver runtimes together with the corresponding comparative performance outcomes based on objective values.

Despite being evaluated under a stricter runtime cap, DAv3c outperforms DAv3 on all three instances and achieves two wins and one loss against the selected heuristics. These results suggest that, for this subset of instances, the additional runtime available to the benchmark solvers does not necessarily lead to superior solution quality.

Instance	TL	DAv3c	DAv3	H	DAv3c vs DAv3	DAv3c vs H
g002750	4161	3614	4256	4161	Win	Loss
g002853	3609	3608	3655	3616	Win	Win
g003153	4055	3613	4080	4055	Win	Win

Table A1: Runtime limits and solver performance for instances with baseline time limits (TL) exceeding 3600 seconds. Here, H means the MQLib Heuristic. DAv3c runtimes are capped at 3600 seconds, while DAv3 and the MQLib Heuristic baseline are evaluated using the original benchmark limits. The final columns report the outcome of DAv3c relative to DAv3 and the heuristic baseline, where "Win" indicates that DAv3c achieves a higher cut value and "Loss" indicates inferior performance.

Appendix B Stability of Solver Outputs

To assess the robustness of solver performance across multiple runs per instance, we compute the coefficient of variation (CV) of the objective values.

Across 3,463 instances, we observe extremely low variability:

- Mean CV: 0.000067
- Median CV: 0
- 75th percentile: 0
- Maximum CV: 0.0101

These results indicate that, for the vast majority of instances, solver outputs are highly stable across runs. Consequently, using the maximum objective value as the representative performance metric does not introduce bias compared to alternative statistics such as the mean or median.

Figure B1 illustrates the distribution of CV values.

Appendix C Distribution of Dominant Edge-Weight Concentration

Figure C1 illustrates the distribution of the n_{90} metric across the integer-weighted benchmark instances. The vertical reference lines indicate the thresholds used to define the concentrated, transitional, and heterogeneous edge-weight categories discussed in Section 3.5.

Appendix D Results on Small and XSmall Instances

The comparison between DAv3c and the best-performing MQLib heuristics on *small* and *xsmall* instances across the size–density categories is shown below. These instances are excluded from the main analysis (Section 4.1.1) because they are generally considered computationally easy and are often solved to near-optimality within very short time limits.

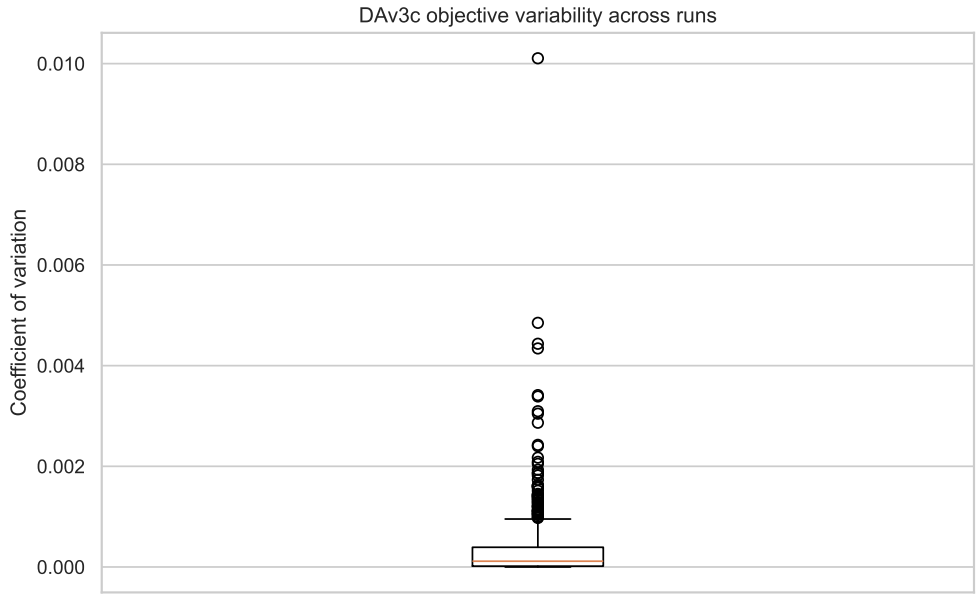


Fig. B1: Distribution of the per-instance coefficients of variation (CVs) for DAv3c. For each graph instance, the CV is computed from the objective values obtained over five independent solver runs, quantifying the run-to-run variability in solution quality. Instances with CV = 0 are excluded for visualization purposes.

The subset consists of a total of 1306 instances. Notably, due to the minimum runtime constraint of DAv3c (see Appendix A), approximately 590 of these instances are solved with a time advantage for DAv3c compared to the heuristics, whose baseline time limits are below one second.

As expected, the results in these regimes are characterized by a high proportion of ties compared to larger instances (Figure D1). This behavior reflects the relative simplicity of the problem instances, where multiple solvers are able to reach similar or identical objective values within the allotted runtime.

Despite this, DAv3c demonstrates a slight overall advantage over the MQLib heuristics across most size–density categories. In particular, DAv3c achieves a higher proportion of wins in the majority of small and xsmall regimes, indicating that its optimization dynamics remain effective even under reduced problem sizes.

An exception is observed in the *xsmall–dense* category, where the performance of DAv3c and the MQLib heuristics is nearly identical. In this regime, DAv3c achieves a win rate of 20.3%, compared to 20.9% for the heuristics, suggesting that both approaches are highly competitive and no clear advantage can be established.

Overall, these findings are consistent with expectations for easy instances: performance differences between solvers are reduced, ties become more frequent, and both approaches are able to reliably reach high-quality solutions.

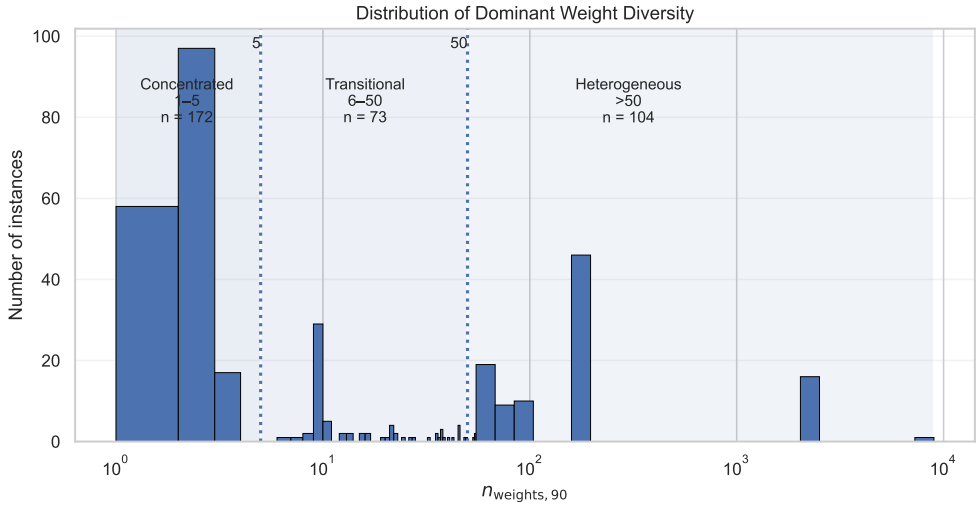


Fig. C1: Distribution of the n_{90} metric across integer-weighted benchmark instances.

Appendix E Graph Metric Definitions

To characterize structural differences between graph categories, we compute several graph-theoretic metrics using the `NetworkX` library.

- **Average Clustering:** Measures the tendency of nodes to form locally connected neighborhoods. It is computed as the average clustering coefficient across all nodes in the graph. Higher values indicate stronger local clustering and the presence of densely interconnected regions.
- **Transitivity:** Measures the global density of triangles in the graph and is defined as the ratio of closed triplets to all connected triplets. Unlike average clustering, transitivity captures triangle structure at the graph level rather than around individual nodes.
- **Cyclomatic Number:** Measures the number of independent cycles in a graph and is computed as

  $$m - n + c,$$

  where m is the number of edges, n is the number of nodes, and c is the number of connected components. Larger values indicate more complex cycle structure and greater redundancy in connectivity.
- **Bridge Fraction:** Measures the fraction of edges whose removal increases the number of connected components. Bridges are edges that do not belong to any cycle. Higher bridge fractions indicate more weakly connected or modular graph structures.
- **Negative Edge Fraction:** Measures the fraction of edges with negative weights relative to the total number of edges in the graph. Higher values indicate a greater prevalence of competing edge interactions in the Max-Cut objective.

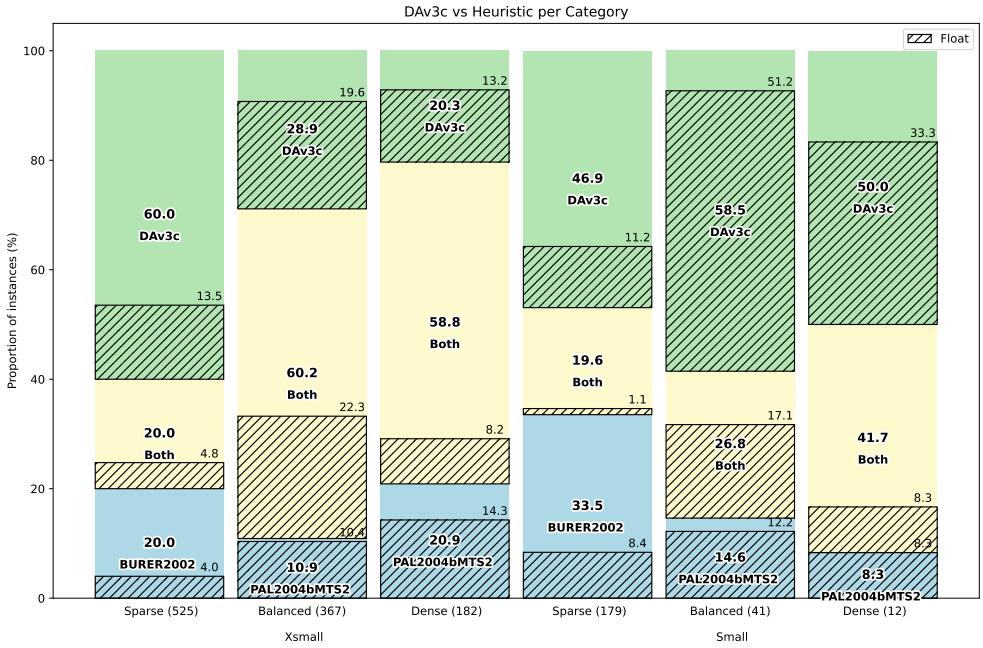


Fig. D1: Performance comparison between DAv3c and the category-selected MQLib heuristics for *xsmall* and *small* instances using the same visualization scheme as Figure 41.

Appendix F Statistical Significance Across Size–Density Categories

This section reports the results of the Wilcoxon signed-rank tests performed on a per-category basis across the size–density groups. For each solver pair, we compare objective values obtained on the same instances using paired samples. The reported p-values test the null hypothesis that the median difference between paired observations is zero, while the effect size r provides a measure of the magnitude of the difference.

Overall, the results support the trends observed in the main analysis. Significant differences are observed in most categories, particularly for comparisons involving DAv2 and DAv3, where effect sizes are often moderate to large. In contrast, the comparison against heuristics exhibits a mix of significant and non-significant outcomes, reflecting the more balanced performance observed in the main results.

DAv3c vs. Heuristics

Table F1 summarizes the category-wise results. Significant differences are observed in most categories, particularly in sparse and dense regimes, while *medium–balanced* does not show a statistically significant difference. Effect sizes range from small to moderate, indicating that while differences exist, they are generally less pronounced than in comparisons with other DA variants.

Category	n	p-value	r	Result
medium_sparse	255	2.81×10^{-2}	0.138	Sig.
medium_balanced	65	9.09×10^{-1}	0.014	Not Sig.
medium_dense	38	1.20×10^{-2}	0.403	Sig.
large_sparse	165	1.86×10^{-9}	0.468	Sig.
large_balanced	103	4.45×10^{-5}	0.402	Sig.
large_dense	33	1.10×10^{-4}	0.636	Sig.
xlarge_sparse	81	1.96×10^{-6}	0.529	Sig.

Table F1: Wilcoxon signed-rank test results for DAv3c and the selected MQLib heuristics.

DAv3c vs. DAv3

Table F2 shows that all categories exhibit statistically significant differences. Effect sizes are consistently moderate to large, with particularly strong effects in dense and balanced regimes. This confirms the systematic performance gap observed in the main results.

Category	n	p-value	r	Result
medium_sparse	236	7.40×10^{-14}	0.487	Sig.
medium_balanced	64	5.21×10^{-5}	0.506	Sig.
medium_dense	38	4.63×10^{-7}	0.740	Sig.
large_sparse	160	2.80×10^{-4}	0.287	Sig.
large_balanced	96	5.02×10^{-11}	0.671	Sig.
large_dense	31	2.21×10^{-6}	0.764	Sig.
xlarge_sparse	81	4.99×10^{-5}	0.451	Sig.

Table F2: Wilcoxon signed-rank test results for DAv3c and DAv3.

DAv3c vs. DAv2

The results in Table F3 indicate that most categories exhibit statistically significant differences, although some exceptions appear in larger instances. In particular, *large–sparse* and *large–balanced* do not show significant differences, with very small effect sizes.

Appendix G Cut-Ratio Distribution: DAv3c vs. DAv3

Figure G1 shows the distribution of cut-value ratios between DAv3c and DAv3, defined as ratio $= \frac{\text{DAv3c}}{\text{DAv3}}$. A ratio above 1 indicates that DAv3c achieves a higher cut value, while a ratio below 1 indicates inferior performance. A ratio of exactly 1 corresponds to a tie.

Category	n	p-value	r	Result
medium_sparse	244	6.42×10^{-12}	0.440	Sig.
medium_balanced	69	4.75×10^{-2}	0.239	Sig.
medium_dense	44	6.48×10^{-5}	0.575	Sig.
large_sparse	162	4.75×10^{-1}	0.056	Not Sig.
large_balanced	113	8.49×10^{-2}	0.162	Not Sig.
large_dense	37	6.86×10^{-3}	0.438	Sig.

Table F3: Wilcoxon signed-rank test results for DAv3c and DAv2.

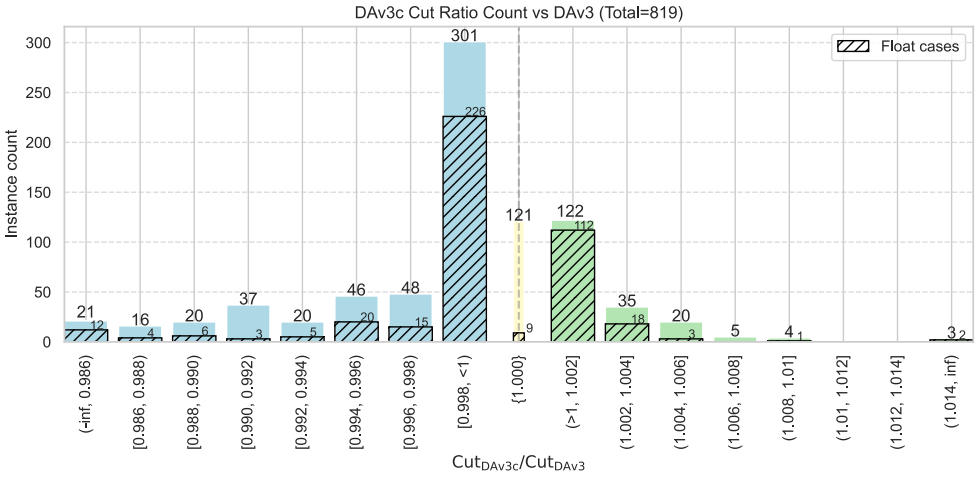


Fig. G1: Distribution of cut-value ratios for DAv3c relative to DAv3 using the same ratio definition as Figure 42.

The distribution is strongly concentrated around 1. A majority of instances (62.15%) fall below 1, indicating that DAv3c is generally outperformed by DAv3. However, a large fraction of these losses are near-optimal. In particular, 36.75% of all instances lie in the interval $[0.998, 1)$, indicating that many losses correspond to very small differences in objective value.

Ties account for 14.77% of instances. On the winning side, improvements are typically small, with the largest concentration in the $(1, 1.002]$ interval (14.90%), and very few instances exceeding a ratio of 1.01.

Overall, the distribution suggests that although DAv3 achieves superior performance on most instances, the gap is often small, and large improvements by DAv3c are rare.

Appendix H Cut-Ratio Distribution: DAv3c vs. DAv2

Figure H1 shows the distribution of cut-value ratios between DAv3c and DAv2, defined as ratio $= \frac{\text{DAv3c}}{\text{DAv2}}$. A ratio above 1 indicates that DAv3c achieves a higher cut value, while a ratio below 1 indicates inferior performance. A ratio of exactly 1 corresponds to a tie.

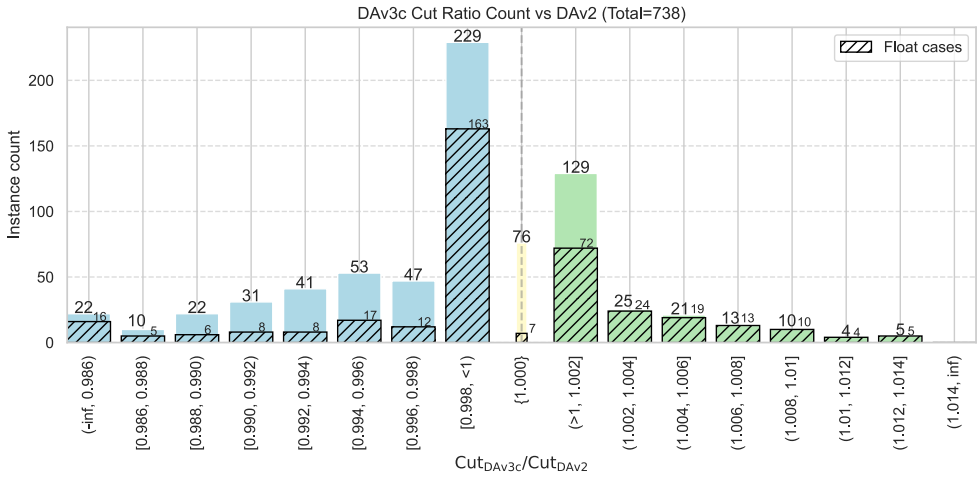


Fig. H1: Distribution of cut-value ratios for DAv3c relative to DAv2 using the same ratio definition as Figure 42.

The distribution is heavily concentrated around 1. While 61.66% of instances fall below 1, indicating that DAv3c is outperformed by DAv2, a substantial portion of these losses are near-optimal. Specifically, 31.03% of all instances lie in the interval $[0.998, 1)$, showing that many losses correspond to very small differences in objective value.

Ties account for 10.30% of instances. On the winning side, the majority of improvements are small, with 17.48% of instances falling in the interval $(1, 1.002]$, and only a small fraction exceeding a ratio of 1.01.

Overall, the distribution indicates that although DAv2 achieves superior performance on most instances, the gap is often small, and large improvements by DAv3c are relatively rare.

Appendix I Statistical Validation of Edge-Weight Representation Analysis

To further validate the observations from the edge-weight representation analysis, we performed paired Wilcoxon signed-rank tests comparing the objective values obtained

by DAv2, DAv3, and DAv3c against the corresponding MQLib heuristic baselines on both integer-weight and non-integer-weight instances.

The Wilcoxon signed-rank test was selected as a non-parametric alternative to the paired t-test, as the objective-value differences do not necessarily follow a normal distribution and may contain large outliers due to variations in graph size and edge-weight magnitudes. The null hypothesis assumes that the median paired difference between the solver and heuristic objective values is zero.

All tests were conducted using a significance level of $\alpha = 0.05$. The results are summarized in Table I1.

Solver	Weight Type	n	p-value	r	Result
DAv2	Integer	289	5.48×10^{-20}	0.538	Sig.
DAv2	Float	360	1.36×10^{-8}	0.299	Sig.
DAv3	Integer	298	1.18×10^{-20}	0.540	Sig.
DAv3	Float	378	2.98×10^{-5}	0.215	Sig.
DAv3c	Integer	305	9.53×10^{-1}	0.003	Not Sig.
DAv3c	Float	435	6.43×10^{-1}	0.022	Not Sig.

Table I1: Wilcoxon signed-rank test results comparing DA solvers against the MQLib heuristics across integer and float edge-weight instances. Statistical significance is evaluated at $\alpha = 0.05$.

The results indicate that the performance differences between DAv2/DAv3 and the heuristic baselines are statistically significant for both integer-weight and non-integer-weight instances. In contrast, DAv3c does not exhibit statistically significant differences from the heuristic baseline under the same evaluation setting. This observation is consistent with the win–tie–loss analysis presented in the main results section, where DAv3c demonstrated substantially higher tie frequencies relative to DAv2 and DAv3.

Appendix J Integer and Non-Integer Instances Across Size–Density Categories

The following figures show the solver-wise win–tie–loss distributions for integer- and non-integer-weighted instances across the individual size–density categories. They complement the aggregated comparison in the main text by providing the corresponding results for each graph category.

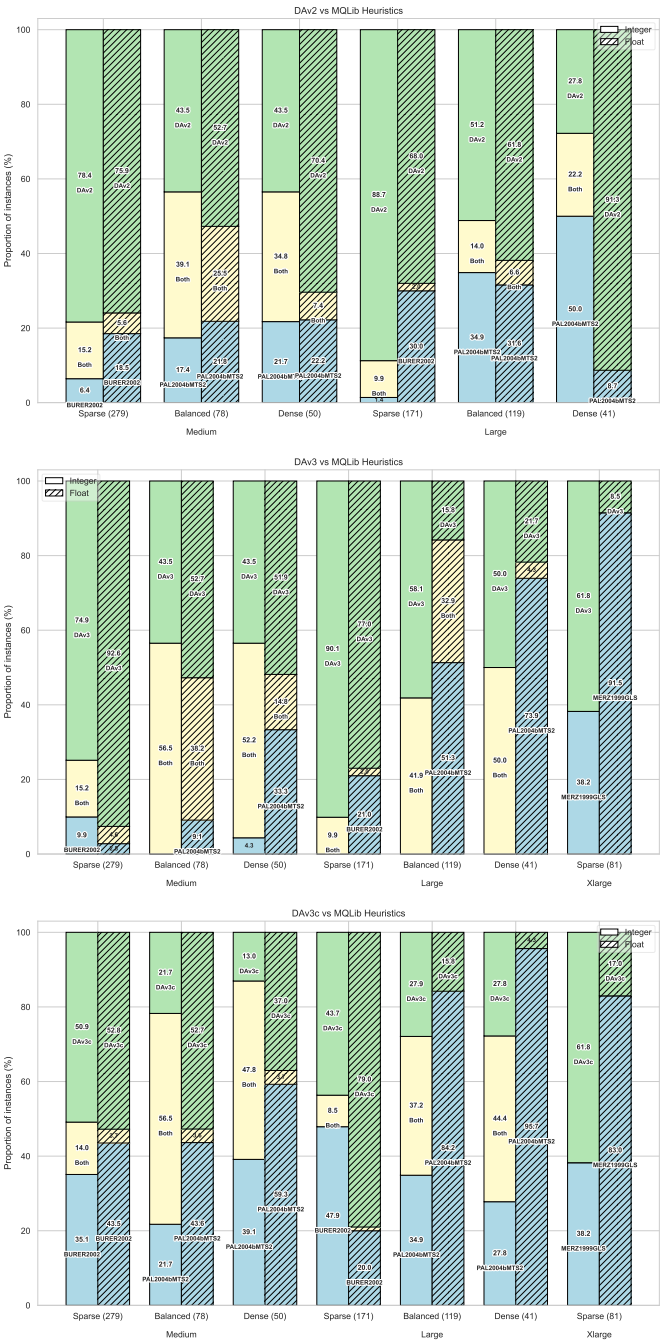


Fig. J1: Win–tie–loss distributions of DAv2, DAv3, and DAv3c against MQLib heuristics on integer and non-integer-weighted instances across size–density categories. Top to bottom: DAv2, DAv3, and DAv3c.

References

[1] Glover, F., Kochenberger, G., Du, Y.: Applications and computational advances for solving the QUBO model. In: Punnen, A.P. (ed.) The Quadratic Unconstrained Binary Optimization Problem. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-04520-2_2

[2] Aaronson, S.: The limits of quantum computers. Scientific American **298**(3), 62–69 (2008)

[3] Lucas, A.: Ising formulations of many NP problems. Frontiers in Physics **2**, 5 (2014) https://doi.org/10.3389/fphy.2014.00005

[4] Mohseni, N., McMahon, P.L., Byrnes, T.: Ising machines as hardware solvers of combinatorial optimization problems. Nature Reviews Physics **4**(6), 363–379 (2022) https://doi.org/10.1038/s42254-022-00440-8

[5] Shaglel, S., Kirsch, M., Winkler, M., Münch, C., Walter, S., Schinkel, F., Kliesch, M.: A comprehensive benchmark of an ising machine on the Max-Cut problem (2025) https://doi.org/10.48550/arXiv.2507.22117 arXiv:2507.22117 [quant-ph]

[6] Dunning, I., Gupta, S., Silberholz, J.: MQLib: A collection of Max-Cut and QUBO heuristics. Accessed: 2026-05-03 (2018). https://github.com/MQLib/MQLib

[7] Karp, R.M.: Reducibility among combinatorial problems. In: Jünger, M., Reinelt, G., Rinaldi, G. (eds.) 50 Years of Integer Programming 1958–2008, pp. 219–241. Springer, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-540-68279-0_8

[8] Punnen, A.P. (ed.): The Quadratic Unconstrained Binary Optimization Problem: Theory, Algorithms, and Applications. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-04520-2

[9] Yamaoka, M., Yoshimura, K., Hayashi, M., Okuyama, T., Aoki, H., Mizuno, H.: A 20k-spin Ising chip to solve combinatorial optimization problems with CMOS annealing. In: Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC), pp. 512–513 (2016). https://doi.org/10.1109/ISSCC.2016.7418120

[10] Choi, M.C.H.: Improved metropolis–hastings algorithms via landscape modification with applications to simulated annealing and the curie–weiss model. Advances in Applied Probability **56**(2), 587–620 (2024) https://doi.org/10.1017/apr.2023.35

[11] Cohn, H., Fielding, M.: Simulated annealing: Searching for an optimal temperature schedule. SIAM Journal on Optimization **9**(3), 779–802 (1999) https://doi.org/10.1137/S1052623497329683

[12] Chen, S., Rosenthal, J.S., Dote, A., Tamura, H., Sheikholeslami, A.: Optimization via rejection-free partial neighbor search. Statistics and Computing **33**, 131 (2023) https://doi.org/10.1007/s11222-023-10258-8

[13] Chen, S., Rosenthal, J.S., Dote, A., Tamura, H., Sheikholeslami, A.: Sampling via rejection-free partial neighbor search. Communications in Statistics – Simulation and Computation **54**, 837–860 (2025) https://doi.org/10.1080/03610918.2023.2266157

[14] Shaglel, S.e.a.: DAMB: Digital Annealer Max-Cut benchmark repository. Accessed: 2026-05-03 (2025). https://github.com/SalwaShaglel/DAMB

[15] Dunning, I., Gupta, S., Silberholz, J.: What works best when? a systematic evaluation of heuristics for Max-Cut and QUBO. INFORMS Journal on Computing **30**(3), 608–624 (2018) https://doi.org/10.1287/ijoc.2017.0791

[16] Shaydulin, R., Lotshaw, P.C., Larson, J., Ostrowski, J., Humble, T.S.: Parameter transfer for quantum approximate optimization of weighted maxcut. ACM Transactions on Quantum Computing **4**(3) (2023) https://doi.org/10.1145/3584706

[17] Toulouse, G.: Theory of the frustration effect in spin glasses: I. Communications on Physics **2**(4), 115–119 (1977)

[18] Kirkpatrick, S.: Frustration and ground-state degeneracy in spin glasses. Physical Review B **16**(10), 4630–4640 (1977)

[19] Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin **1**(6), 80–83 (1945)

[20] Demšar, J.: Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research **7**, 1–30 (2006)

[21] Franco, C., Little, R.J.A., Louis, T.A., Slud, E.V.: Comparative study of confidence intervals for proportions in complex sample surveys. Journal of Survey Statistics and Methodology **7**(3), 334–364 (2019) https://doi.org/10.1093/jssam/smy019

[22] Glover, F.: Tabu search—part i. ORSA Journal on Computing **1**(3), 190–206 (1989)

[23] Glover, F.: Tabu search—part ii. ORSA Journal on Computing **2**(1), 4–32 (1990)